

Rekurze

- objekt nebo jev je definován pomocí sebe sama

V programování se rekurze objevuje ve dvou rovinách:

- **rekurzivní algoritmus** – řešení úlohy je definováno pomocí řešení menších instancí téhož problému (tzn. podúloh stejného charakteru)
- **rekurzivní volání funkce** – funkce volá sama sebe (přímo nebo případně nepřímo prostřednictvím jiných funkcí)

Většinou se rekurzivní algoritmy realizují pomocí rekurzivních volání, ale není to nezbytné:

- rekurzivní algoritmus lze realizovat bez rekurzivních volání (pomocí vlastního zásobníku na uložení rozpracovaných nedokončených podúloh)
→ více práce pro programátora, program obvykle delší a méně přehledný, výpočet ale může být o něco efektivnější
- rekurzivní volání lze teoreticky použít i při realizaci nerekurzivních iteračních algoritmů (dokonce každý cyklus lze nahradit rekurzivní funkcí)
→ zpravidla nevhodné, nečitelný a méně efektivní program

Ukončení rekurze

- rekurzivní volání funkce musí být vázáno na nějakou podmínku, která časem jistě přestane platit → jinak „zacyklení výpočtu“
- na rozdíl od nekonečného while-cyklu nastane v Pythonu výjimka `RecursionError` po dosažení maximální povolené hloubky rekurze (default 1000, v IDLE zvýšeno na 1013)

```
def rekurze(p) :  
    print(p)  
    rekurze(p+1)  
  
rekurze(1)
```

Nastavení limitu hloubky rekurze v Pythonu:

```
from sys import setrecursionlimit  
setrecursionlimit(10000)
```

Poznámka: programovací jazyky bez limitu na hloubku rekurze
→ běhová chyba přetečení zásobníku (stack overflow)

Průběh výpočtu při rekurzivním volání funkce

- najednou je rozpočítáno více exemplářů téže funkce
- všechny počítají podle téhož kódu
- každý exemplář má na volacím zásobníku svůj vlastní aktivační záznam s lokálními proměnnými, parametry a technickými údaji (kde je rozpočítán, návratová adresa)
- funkce nemá přístup k lokálním proměnným jiného rekurzivního exempláře

Příklad 3: výpis znaků ze vstupu pozpátku

```
def otoc() :  
    u = input("Znak: ")  
    if u != " "  
        otoc()  
    print(u)
```

Vstup: A
B
C
<mezera>

Výstup: <mezera>
C
B
A

Příklady jednoduchých rekurzivních funkcí

Palindrom – řetězec se čte stejně zleva i zprava (je symetrický)

```
def palindrom1(s):  
    n = len(s)  
    for i in range(n//2):  
        if s[i] != s[n-i-1]:  
            return False  
    return True
```

```
def palindrom2(s):  
    if len(s) <= 1 :  
        return True  
    else:  
        return s[0] == s[-1] and palindrom2(s[1:-1])
```

Eukleidův algoritmus (odčítací) realizovaný cyklem (*bylo dříve*):

```
def nsd(x, y):  
    while x != y:  
        if x > y:  
            x -= y  
        else:  
            y -= x  
    return x
```


Eukleidův algoritmus realizovaný rekurzivní funkcí
- přesně kopíruje rekurzivní vztah pro NSD,
na němž je Eukleidův algoritmus založen:

když $X > Y$	$\text{NSD}(X, Y) = \text{NSD}(X-Y, Y)$
když $X < Y$	$\text{NSD}(X, Y) = \text{NSD}(X, Y-X)$
když $X = Y$	$\text{NSD}(X, Y) = X$

```
def nsd(x, y):  
    if x > y:  
        return nsd(x-y, y)  
    elif x < y:  
        return nsd(x, y-x)  
    else:  
        return x
```

Eukleidův algoritmus realizovaný rekurzivní funkcí
- zrychlená varianta s operací modulo

```
def nsd(x, y):  
    if y == 0:  
        return x  
    return nsd(y, x % y)
```

Faktoriál $n!$ (součin čísel od 1 do n , pro $n \geq 0$)

rekurzivní definice: $n! = 1$ pro $n = 0$
 $n! = n \cdot (n-1)!$ pro $n > 0$

```
def faktorial(n):  
    f = 1  
    for i in range(2, n+1):  
        f *= i  
    return f
```

```
def faktorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * faktorial(n-1)
```

v obou případech časová složitost $O(n)$

Fibonacciho čísla

$$\begin{aligned} F_0 &= 0 \\ F_1 &= 1 \\ F_n &= F_{n-1} + F_{n-2} \quad \text{pro } n > 1 \end{aligned}$$

rekurzivní definice posloupnosti čísel

→ realizace rekurzivní funkcí přesně podle definice:

```
def fib(n) :  
    if n == 0 or n == 1:  
        return n  
    else:  
        return fib(n-1) + fib(n-2)
```

funkce teoreticky správná, ale časová složitost exponenciální

→ pro $n > \text{cca } 40$ prakticky nepoužitelná

důvod: mnohokrát se opakovaně počítají stejné věci

Možnosti řešení:

1. rekurzivní algoritmus + pomocné pole velikosti n
pro uložení již spočítaných funkčních hodnot

→ každé F_i se počítá jen jednou → časová složitost $O(n)$

„chytrá rekurze“

kešování hodnot (cache = mezipaměť)

memoizace (memory = paměť)

dynamické programování

2. počítat hodnoty iteračně odspodu – v pořadí $F_1, F_2, \dots F_n$
→ časová složitost $O(n)$, navíc stačí konstantní paměť

dynamické programování

```
def fib(n):  
    if n == 0:  
        return 0  
    a = 0; b = 1  
    while n > 1:  
        a, b = b, a+b  
        n -= 1  
    return b
```

3. z rekurzivní definice odvodit explicitní vzorec a počítat podle něj

$$F_n = \frac{\sqrt{5}}{5} \left(\left(\frac{1 + \sqrt{5}}{2} \right)^n - \left(\frac{1 - \sqrt{5}}{2} \right)^n \right)$$

problémy: časová složitost, zaokrouhlovací chyby v typu float

4. využití rychlého umocňování matice

Platí rovnost $\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \cdot \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} b \\ a + b \end{pmatrix}$

Tedy $\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \cdot \begin{pmatrix} F0 \\ F1 \end{pmatrix} = \begin{pmatrix} F1 \\ F2 \end{pmatrix}$

$$\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \cdot \begin{pmatrix} F1 \\ F2 \end{pmatrix} = \begin{pmatrix} F2 \\ F3 \end{pmatrix}$$

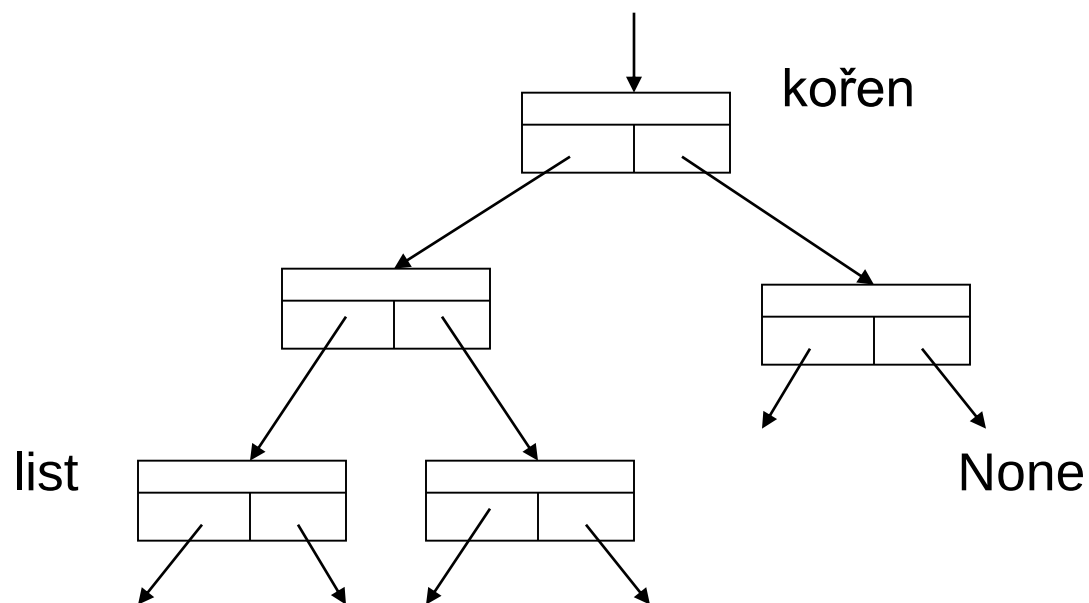
...

$$\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}^n \cdot \begin{pmatrix} F0 \\ F1 \end{pmatrix} = \begin{pmatrix} Fn \\ Fn + 1 \end{pmatrix}$$

Matici $\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}^n$ spočítáme rychlým umocňováním

→ časová složitost $O(\log N)$

Binární strom



```
class Vrchol:
    """vrchol binárního stromu"""

    def __init__(self, x = None, l = None, r = None) :
        self.info = x           # uložená hodnota
        self.levy = l           # levý syn
        self.pravy = r          # pravý syn
```

Výška H binárního stromu o N uzlech

= délka nejdelší cesty z kořenu do listu

minimální výška: vyvážený strom

$$N = 2^0 + 2^1 + \dots + 2^H = 2^{H+1} - 1 \quad \rightarrow H \approx \log_2 N$$

maximální výška: degenerovaný strom $\rightarrow H \approx N$

v průměrném případě výška $O(\log N)$

Příklady použití binárního stromu:

- halda (*bylo*)
- binární vyhledávací strom (*bude příště*)
- reprezentace aritmetického výrazu (*bude později*)

Průchod binárním stromem (do hloubky)

- v každém vrcholu provést zvolenou akci
(např. vypsát hodnotu atributu “info”)

Varianty průchodu podle pořadí zpracování vrcholů:

PREORDER – nejprve zpracuje vrchol, pak jde postupně do obou jeho podstromů

INORDER – nejprve jde do levého podstromu, pak zpracuje vrchol, nakonec jde do pravého podstromu

POSTORDER – nejprve jde postupně do obou podstromů vrcholu, pak zpracuje vrchol samotný

```

class Vrchol:
    """vrchol binárního stromu"""

    def __init__(self, x = None):
        self.info = x                # uložená hodnota
        self.levy = None            # levý syn
        self.pravy = None          # pravý syn

    def preorder(self):
        """průchod stromem s kořenem v tomto vrcholu
           metodou preorder, vypisuje hodnoty všech
           vrcholů
        """
        print(self.info)
        if self.levy != None:
            self.levy.preorder()
        if self.pravy != None:
            self.pravy.preorder()

```

```

def inorder(self):
    """průchod stromem s kořenem v tomto vrcholu
       metodou inorder, vypisuje hodnoty všech
       vrcholů"""
    if self.levy != None:
        self.levy.inorder()
    print(self.info)
    if self.pravy != None:
        self.pravy.inorder()

def postorder(self):
    """průchod stromem s kořenem v tomto vrcholu
       metodou postorder, vypisuje hodnoty všech
       vrcholů"""
    if self.levy != None:
        self.levy.postorder()
    if self.pravy != None:
        self.pravy.postorder()
    print(self.info)

```

Průchod do hloubky bez použití rekurze – pomocí zásobníku

```
DO_ZÁSOBNÍKU(Kořen)
dokud ZÁSOBNÍK není prázdný
    P = ZE_ZÁSOBNÍKU
    AKCE(P.info)
    jestliže P.pravy != None: DO_ZÁSOBNÍKU(P.pravy)
    jestliže P.levy != None: DO_ZÁSOBNÍKU(P.levy)
```

Průchod do šířky (po vrstvách) – pomocí fronty

```
DO_FRONTY(Kořen)
dokud FRONTA není prázdná
    P = Z_FRONTY
    AKCE(P.info)
    jestliže P.levy != None: DO_FRONTY(P.levy)
    jestliže P.pravy != None: DO_FRONTY(P.pravy)
```


Časová složitost všech uvedených metod průchodu:

$O(N)$, kde N je počet vrcholů ve stromu

- neboť každý vrchol stromu je navštíven právě jednou a jeho zpracování má konstantní časovou složitost